

CESAR SCHOOL
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
PROJETO INTEGRADOR — 3º PERÍODO — 2026.2

João Pedro Cavalcanti Souza
Lucas Henrique Gomes Medeiros
Luis Felipe Farias Nunes
Luis Lucena Wanderley G.
Mariana Xavier Bezerra
Matheus Rodrigues Larré
Micaella Maria Barbosa Cabral

ESTRUTURAS DE DADOS DO SISTEMA ROTA VITAL:
implementação em C com gerenciamento manual de memória
e tradução comentada para Java

RECIFE
2026

João Pedro Cavalcanti Souza
Lucas Henrique Gomes Medeiros
Luis Felipe Farias Nunes
Luis Lucena Wanderley G.
Mariana Xavier Bezerra
Matheus Rodrigues Larré
Micaella Maria Barbosa Cabral

ESTRUTURAS DE DADOS DO SISTEMA ROTA VITAL:
implementação em C com gerenciamento manual de memória
e tradução comentada para Java

Relatório técnico apresentado à disciplina de Algoritmos e Estruturas de Dados, como parte da entrega da Unidade 1 do Projeto Integrador do 3º período do curso de Análise e Desenvolvimento de Sistemas da CESAR School.

RESUMO

Este relatório descreve as estruturas de dados que sustentam o domínio do Rota Vital, sistema de gestão e distribuição de hemocomponentes desenvolvido no Projeto Integrador. Foram implementadas, primeiro em linguagem C e com alocação manual de memória, uma lista encadeada para o estoque de bolsas, uma fila para as requisições hospitalares, uma pilha para o histórico de operações do estoque e uma árvore binária de busca para o catálogo de bolsas por identificador. Em seguida, as mesmas estruturas foram reescritas em Java, sobre as entidades já usadas pela aplicação Spring Boot, preservando os algoritmos da versão em C. O texto justifica a escolha de cada estrutura a partir das necessidades do domínio, compara as duas implementações trecho a trecho e apresenta os testes aplicados às duas versões, com os mesmos cenários. Os resultados mostram que as operações de inserção, remoção e consulta se comportam da mesma forma nas duas linguagens e que a versão em C libera toda a memória que aloca.

Palavras-chave: estruturas de dados; listas encadeadas; árvore binária de busca; alocação dinâmica de memória; Java.

SUMÁRIO

1	Introdução	5
2	Objetivos	6
2.1	Objetivo geral	6
2.2	Objetivos específicos	6
3	Modelagem do domínio	7
3.1	Entidades	7
3.2	Escolha das estruturas	7
3.3	Delimitação do escopo	8
4	Metodologia	9
5	Implementação e tradução comentada	10
5.1	Estoque: lista encadeada	10
5.2	Requisições: fila encadeada	12
5.3	Histórico: pilha encadeada	14
5.4	Gestão do estoque: lista e pilha integradas	15
5.5	Catálogo: árvore binária de busca	17
5.6	Síntese das correspondências	19
6	Testes e resultados	21
6.1	Cenários de teste	21
6.2	Resultados	21
6.3	Custo das operações	21
7	Integração com a aplicação	23
8	Considerações finais	24
	REFERÊNCIAS	25
A	Organização dos arquivos e execução	26

1 INTRODUÇÃO

O Rota Vital é uma aplicação web em Java com Spring Boot que apoia a rede de sangue na gestão do estoque de hemocomponentes, no atendimento às requisições dos hospitais e na distribuição das bolsas. O sistema é desenvolvido ao longo do semestre no Projeto Integrador e reúne contribuições de cinco disciplinas, cabendo a Algoritmos e Estruturas de Dados (AED) as estruturas e os algoritmos que organizam os dados do domínio.

Na Unidade 1, a disciplina de AED avalia o raciocínio por trás das estruturas lineares e das árvores binárias, e não apenas o funcionamento do código. Por esse motivo, a entrega exige que cada estrutura seja escrita primeiro em C, com alocação e liberação explícitas de memória (`malloc` e `free`) e manipulação direta de ponteiros, e depois reimplementada em Java, de forma que possa ser usada pela aplicação. Um texto de tradução comentada deve explicar por que a versão Java corresponde à versão em C.

Este relatório cumpre essa exigência. O Capítulo 2 apresenta os objetivos; o Capítulo 3 descreve a modelagem do domínio e justifica a escolha das estruturas; o Capítulo 4 expõe a metodologia de desenvolvimento e de testes; o Capítulo 5, parte central do trabalho, traz a tradução comentada de cada estrutura; o Capítulo 6 apresenta os testes e seus resultados; o Capítulo 7 mostra como a aplicação consome as classes Java; e o Capítulo 8 reúne as considerações finais.

2 OBJETIVOS

2.1 Objetivo geral

Implementar as estruturas básicas de domínio do Rota Vital (estoque, requisições e histórico de operações) em C, com gerenciamento manual de memória, e reimplementá-las em Java, demonstrando a equivalência lógica entre as duas versões.

2.2 Objetivos específicos

- a) modelar as entidades do domínio necessárias às estruturas da Unidade 1;
- b) justificar a escolha de lista, fila, pilha e árvore binária de busca para cada necessidade do sistema;
- c) implementar as operações de inserção, remoção e consulta, incluindo funções recursivas;
- d) reescrever as estruturas em Java sem dependência do framework, para que possam ser usadas pelos serviços da aplicação;
- e) verificar as duas versões com os mesmos cenários de teste e confirmar a ausência de vazamento de memória na versão em C.

3 MODELAGEM DO DOMÍNIO

3.1 Entidades

As estruturas da Unidade 1 manipulam duas entidades do domínio: a bolsa de hemocomponente e a requisição hospitalar. Na aplicação, elas já existiam como entidades JPA (`Bolsa`, `Requisicao` e `ItemRequisicao`). Para facilitar a tradução, as estruturas (`struct`) em C foram definidas a partir dessas classes, com os mesmos nomes de campos e de constantes de enumeração, como mostra a Tabela 3.1.

Tabela 3.1 – Correspondência entre as entidades Java e os tipos em C

Java	C (<code>dominio.h</code>)	Observação
<code>Bolsa</code>	<code>Bolsa</code>	Mesmos campos: identificador, componente, tipo sanguíneo, datas de coleta e validade, situação. <code>LocalDate</code> foi representado por <code>struct Data</code> . Os relacionamentos JPA (doador e estoque) não são usados pelas estruturas e ficaram de fora.
<code>Requisicao</code> <code>ItemRequisicao</code>	e <code>Requisicao</code>	Em C, a requisição carrega um único item (componente, tipo e quantidade). Em Java, a fila guarda a entidade completa, com sua lista de itens.
<code>HemoComponente</code> , <code>TipoSanguineo</code> , <code>StatusBolsa</code> , <code>Prioridade</code>	enumerações homônimas	Mesmas constantes e na mesma ordem.

Fonte: Elaborado pelos autores (2026).

3.2 Escolha das estruturas

O estoque do hemocentro muda de tamanho continuamente: doações entram, bolsas saem para hospitais e outras são descartadas, e não há um limite conhecido em tempo de compilação. Um vetor exigiria realocação e cópia dos elementos ao crescer, além de deslocamento ao remover uma bolsa do meio. Por isso optou-se por uma **lista simplesmente encadeada**, na qual cada bolsa ocupa um nó alocado individualmente e a remoção se resume a religar ponteiros (TENENBAUM; LANGSAM; AUGENSTEIN, 1995). A lista mantém um ponteiro para o último nó, o que permite inserir no fim sem percorrê-la.

As requisições dos hospitais devem ser atendidas na ordem em que chegam, o que caracteriza a disciplina FIFO (*first in, first out*). Uma **fila encadeada** com ponteiros para o início e para o fim realiza a inserção e a retirada em tempo constante (CORMEN *et al.*, 2012).

O histórico de operações do estoque tem duas finalidades: auditoria e a possibilidade de

desfazer a última operação registrada por engano. Como desfazer sempre atua sobre a operação mais recente, a estrutura adequada é a **pilha** (LIFO, *last in, first out*), também encadeada.

Por fim, consultar uma bolsa pelo identificador na lista exige percorrê-la, com custo proporcional ao número de bolsas. Para essa consulta foi implementado um catálogo em **árvore binária de busca** (ABB), em que cada comparação descarta uma subárvore inteira, e cujo percurso em ordem simétrica lista as bolsas ordenadas pelo identificador (ZIVIANI, 2011). A ABB foi também o lugar natural para exercitar a recursão: inserção, busca, remoção, percurso, altura e liberação da memória são recursivos.

3.3 Delimitação do escopo

Conforme o enunciado da Unidade 1, roteirização, priorização por validade (FEFO), índice por *hash* e compatibilidade ABO/Rh ficam para a Unidade 2. Por isso a lista do estoque mantém as bolsas na ordem de chegada, sem ordená-las por validade, e a fila de requisições é FIFO pura: a prioridade (urgente ou normal) é apenas um atributo da requisição e não altera a ordem de atendimento nesta etapa.

4 METODOLOGIA

O trabalho seguiu três etapas. Na primeira, cada estrutura foi implementada em C (padrão C99, compilador GCC), em um par de arquivos de cabeçalho e de implementação, com alocação dinâmica por meio de `malloc` e `free` (KERNIGHAN; RITCHIE, 1989) e testes próprios escritos em C. Na segunda, a estrutura foi reescrita em Java 17, no pacote `com.hemorede.estruturas` da aplicação, mantendo os mesmos nomes de campos e a mesma sequência de passos de cada operação. Na terceira, os cenários de teste da versão em C foram reproduzidos em JUnit 5, de modo que as duas versões fossem verificadas com as mesmas entradas e os mesmos resultados esperados.

Na versão em C, os testes foram executados de duas formas: compilados com o AddressSanitizer, que interrompe a execução diante de acesso a memória já liberada, e sob o Valgrind, que contabiliza as alocações e liberações e acusa blocos não liberados ao final. Na versão em Java, além dos testes da Unidade 1, foi executada a suíte completa da aplicação, para confirmar que as novas classes não interferem no restante do sistema.

Uma decisão de projeto orientou toda a implementação em Java: as estruturas são encadeadas manualmente, por meio de classes internas que representam os nós, sem uso de `ArrayList`, `LinkedList`, `ArrayDeque` ou `TreeMap`. Usar essas coleções esconderia exatamente a lógica que a disciplina pede para demonstrar e quebraria a correspondência com o código em C.

O código-fonte está no repositório do projeto¹, organizado conforme o Apêndice A.

¹https://github.com/Luisr-nunes/projeto_Rota_Vital

5 IMPLEMENTAÇÃO E TRADUÇÃO COMENTADA

Este capítulo apresenta, para cada estrutura, o trecho central do código em C, o trecho correspondente em Java e a explicação da equivalência entre eles. Os trechos foram reproduzidos dos arquivos do repositório, com os comentários resumidos.

5.1 Estoque: lista encadeada

A lista é composta por nós do tipo NoBolsa, cada um com uma bolsa e um ponteiro para o próximo nó, e por um descritor ListaEstoque com os ponteiros inicio e fim e o contador tamanho. O Código 5.1 mostra a inserção e a remoção em C.

Código 5.1 – Inserção e remoção na lista do estoque, em C (lista_estoque.c)

```

1  int lista_inserir(ListaEstoque *lista, Bolsa bolsa) {
2      NoBolsa *novo;
3      if (lista_buscar(lista, bolsa.id) != NULL)
4          return LISTA_DUPLICADA;
5      novo = (NoBolsa *) malloc(sizeof(NoBolsa));
6      if (novo == NULL)
7          return LISTA_SEM_MEMORIA;
8      novo->bolsa = bolsa;
9      novo->proximo = NULL;
10     if (lista->fim == NULL)
11         lista->inicio = novo;
12     else
13         lista->fim->proximo = novo;
14     lista->fim = novo;
15     lista->tamanho++;
16     return LISTA_OK;
17 }
18
19 int lista_remover(ListaEstoque *lista, long id, Bolsa *removida) {
20     NoBolsa *anterior = NULL;
21     NoBolsa *atual = lista->inicio;
22     while (atual != NULL && atual->bolsa.id != id) {
23         anterior = atual;
24         atual = atual->proximo;
25     }
26     if (atual == NULL)
27         return 0;
28     if (anterior == NULL)
29         lista->inicio = atual->proximo;
30     else
31         anterior->proximo = atual->proximo;
32     if (atual == lista->fim)
33         lista->fim = anterior;
34     if (removida != NULL)

```

```

35     *removida = atual->bolsa;  /* copia antes de liberar */
36     free(atual);
37     lista->tamanho--;
38     return 1;
39 }

```

Fonte: Elaborado pelos autores (2026).

Código 5.2 – Inserção e remoção na lista do estoque, em Java (ListaEstoque.java)

```

1  public boolean inserir(Bolsa bolsa) {
2      validar(bolsa);
3      if (buscar(bolsa.getId()) != null)
4          return false;
5      NoBolsa novo = new NoBolsa(bolsa);
6      if (fim == null)
7          inicio = novo;
8      else
9          fim.proximo = novo;
10     fim = novo;
11     tamanho++;
12     return true;
13 }
14
15 public Bolsa remover(long id) {
16     NoBolsa anterior = null;
17     NoBolsa atual = inicio;
18     while (atual != null && atual.bolsa.getId() != id) {
19         anterior = atual;
20         atual = atual.proximo;
21     }
22     if (atual == null)
23         return null;
24     if (anterior == null)
25         inicio = atual.proximo;
26     else
27         anterior.proximo = atual.proximo;
28     if (atual == fim)
29         fim = anterior;
30     atual.proximo = null;
31     tamanho--;
32     return atual.bolsa;
33 }

```

Fonte: Elaborado pelos autores (2026).

A classe interna NoBolsa, com os campos bolsa e proximo, corresponde ao struct NoBolsa, e os campos inicio, fim e tamanho da classe ListaEstoque são os mesmos do descritor em C. Na inserção, a expressão `new NoBolsa(bolsa)` cumpre o papel do `malloc` seguido das atribuições `novo->bolsa = bolsa` e `novo->proximo = NULL`, já que o construtor guarda a bolsa e o Java inicializa a referência proximo com null. O encadeamento no fim segue a mesma

sequência nas duas linguagens: se a lista está vazia, o novo nó passa a ser o início; caso contrário, o último nó passa a apontar para ele; em ambos os casos, fim é atualizado. A verificação de identificador duplicado também acontece antes da alocação nas duas versões, mudando apenas a forma de sinalizar o resultado (código LISTA_DUPLICADA em C e false em Java).

A remoção percorre a lista com dois ponteiros, anterior e atual, e trata os mesmos casos: se o nó removido é o primeiro, o início avança; se está no meio ou no fim, o nó anterior passa a apontar para o sucessor do removido; e, se o removido era o último, fim recua para anterior. Esse último ajuste é verificado explicitamente nos testes das duas versões, que removem a bolsa do fim e conferem qual nó passou a ser o último. A diferença está na liberação da memória. Em C, o nó precisa ser liberado com `free(atual)`, e por isso a bolsa é copiada para o parâmetro de saída `*removida` antes dessa chamada, pois acessar o nó depois dela seria uso de memória liberada. Em Java não há liberação explícita: depois que o nó deixa de ser referenciado pela lista, ele se torna inalcançável e é recolhido pelo coletor de lixo (ORACLE, 2021), o que permite devolver a bolsa diretamente. O caso de falta de memória, tratado em C pelo teste do retorno de `malloc`, não tem equivalente no código Java, porque a própria máquina virtual lança `OutOfMemoryError`.

Há uma diferença de semântica que foi mantida de forma deliberada. Em C, o nó guarda uma cópia da bolsa, já que a atribuição entre estruturas copia o valor. Em Java, o nó guarda uma referência ao objeto recebido. Optou-se por não clonar a entidade em Java porque ela é gerenciada pelo JPA: interessa à aplicação que o objeto mantido na estrutura seja o mesmo objeto persistido, para que uma alteração de situação gravada pelo repositório seja vista também pela lista.

A consulta preserva o mesmo comportamento nas duas linguagens. A função `lista_buscar` devolve um ponteiro para a bolsa dentro do nó, de modo que alterar a situação da bolsa por meio dele altera o próprio estoque; o método `buscar` devolve a referência ao mesmo objeto guardado, com o mesmo efeito. A lista oferece ainda uma contagem recursiva das bolsas disponíveis de um tipo e componente, cujo caso base é o fim da lista (ponteiro nulo, resultado zero) e cujo passo soma um, se o nó atual atende ao critério, à contagem do restante da lista.

5.2 Requisições: fila encadeada

A fila usa a mesma organização em nós, com acesso restrito às extremidades: inserção pelo fim e retirada pelo início.

Código 5.3 – Enfileirar e desenfileirar requisições, em C (`fila_requisicoes.c`)

```

1 int fila_enfileirar(FilaRequisicoes *fila, Requisicao requisicao) {
2     NoRequisicao *novo = (NoRequisicao *) malloc(sizeof(NoRequisicao));
3     if (novo == NULL)
4         return 0;
5     novo->requisicao = requisicao;

```

```

6     novo->proximo = NULL;
7     if (fila->fim == NULL)
8         fila->inicio = novo;
9     else
10        fila->fim->proximo = novo;
11    fila->fim = novo;
12    fila->tamanho++;
13    return 1;
14 }
15
16 int fila_desenfileirar(FilaRequisicoes *fila, Requisicao *saida) {
17     NoRequisicao *removido;
18     if (fila->inicio == NULL)
19         return 0;
20     removido = fila->inicio;
21     if (saida != NULL)
22         *saida = removido->requisicao;
23     fila->inicio = removido->proximo;
24     if (fila->inicio == NULL)
25         fila->fim = NULL;
26     free(removido);
27     fila->tamanho--;
28     return 1;
29 }

```

Fonte: Elaborado pelos autores (2026).

Código 5.4 – Enfileirar e desenfileirar requisições, em Java (FilaRequisicoes.java)

```

1 public void enfileirar(Requisicao requisicao) {
2     Objects.requireNonNull(requisicao, "requisicao nao pode ser nula");
3     NoRequisicao novo = new NoRequisicao(requisicao);
4     if (fim == null)
5         inicio = novo;
6     else
7         fim.proximo = novo;
8     fim = novo;
9     tamanho++;
10 }
11
12 public Requisicao desenfileirar() {
13     if (inicio == null)
14         return null;
15     NoRequisicao removido = inicio;
16     inicio = removido.proximo;
17     if (inicio == null)
18         fim = null;
19     removido.proximo = null;
20     tamanho--;
21     return removido.requisicao;
22 }

```

Fonte: Elaborado pelos autores (2026).

As duas versões realizam enfileiramento e desenfileiramento sem percorrer a fila, em tempo constante, porque mantêm os ponteiros para as duas extremidades. O método `enfileirar` reproduz a lógica de `fila_enfileirar`: com a fila vazia, o novo nó é ao mesmo tempo início e fim; caso contrário, é ligado após o último. O ponto que exige mais cuidado está no desenfileiramento, na instrução que volta `fim` para nulo quando a fila se esvazia. Em C, sem essa instrução, `fim` continuaria apontando para o nó que acabou de ser liberado por `free(removido)`, e o próximo enfileiramento escreveria em memória inválida ao executar `fila->fim->proximo = novo`. Em Java não ocorreria acesso inválido, mas o efeito seria igualmente incorreto: `fim` ainda referenciaria o nó antigo, o novo nó seria ligado a ele, `inicio` permaneceria nulo e a requisição seria perdida. Por isso a instrução foi mantida na tradução, e os testes das duas versões esvaziavam a fila e voltam a enfileirar para conferir esse caminho.

A fila vazia é sinalizada em C pelo retorno zero, com a requisição devolvida por um parâmetro de saída, e em Java pelo retorno `null`, já que o método devolve diretamente a referência. As consultas por identificador e por posição na fila são percursos sequenciais idênticos nas duas linguagens e não alteram a ordem de atendimento.

5.3 Histórico: pilha encadeada

A pilha mantém apenas o ponteiro para o topo, e cada nó aponta para o nó imediatamente abaixo. Cada elemento é uma operação do estoque, formada pelo tipo (entrada ou saída) e pela bolsa envolvida.

Código 5.5 – Empilhar e desempilhar operações, em C (`pilha_historico.c`)

```

1 int pilha_empilhar(PilhaHistorico *pilha, OperacaoEstoque operacao) {
2     NoOperacao *novo = (NoOperacao *) malloc(sizeof(NoOperacao));
3     if (novo == NULL)
4         return 0;
5     novo->operacao = operacao;
6     novo->abaixo = pilha->topo;
7     pilha->topo = novo;
8     pilha->tamanho++;
9     return 1;
10 }
11
12 int pilha_desempilhar(PilhaHistorico *pilha, OperacaoEstoque *saida) {
13     NoOperacao *removido;
14     if (pilha->topo == NULL)
15         return 0;
16     removido = pilha->topo;
17     if (saida != NULL)
18         *saida = removido->operacao;
19     pilha->topo = removido->abaixo;
20     free(removido);

```

```

21     pilha->tamanho--;
22     return 1;
23 }

```

Fonte: Elaborado pelos autores (2026).

Código 5.6 – Empilhar e desempilhar operações, em Java (PilhaHistorico.java)

```

1 public void empilhar(OperacaoEstoque operacao) {
2     Objects.requireNonNull(operacao, "operacao nao pode ser nula");
3     topo = new NoOperacao(operacao, topo);
4     tamanho++;
5 }
6
7 public OperacaoEstoque desempilhar() {
8     if (topo == null)
9         return null;
10    NoOperacao removido = topo;
11    topo = removido.abaxio;
12    tamanho--;
13    return removido.operacao;
14 }

```

Fonte: Elaborado pelos autores (2026).

Em C, empilhar envolve três passos explícitos: alocar o nó, fazê-lo apontar para o topo atual e torná-lo o novo topo. Em Java, a instrução `topo = new NoOperacao(operacao, topo)` realiza os mesmos três passos em uma linha. A expressão à direita é avaliada antes da atribuição, de modo que o construtor recebe o topo antigo e o guarda no campo `abaixo`, e só depois a referência `topo` passa a apontar para o novo nó. Como um nó nunca muda de vizinho depois de empilhado, o campo `abaixo` pôde ser declarado `final` em Java. O desempilhamento é simétrico: guarda-se o nó do topo, o topo desce para o nó de baixo e o nó retirado é liberado, com `free` em C e pela perda da última referência em Java. A operação registrada é uma estrutura em C e um `record` em Java. O `record` é imutável, o que corresponde ao fato de que, em C, a operação é copiada para dentro do nó e não é mais alterada.

5.4 Gestão do estoque: lista e pilha integradas

As entradas e saídas de bolsas não são feitas diretamente na lista, mas por um módulo de gestão que aplica a operação na lista e a registra na pilha. Esse módulo também implementa a operação de desfazer.

Código 5.7 – Registro de saída e desfazer, em C (gestao_estoque.c)

```

1 int estoque_registrar_saida(GestaoEstoque *g, long id, Bolsa *saiu) {
2     OperacaoEstoque op;
3     Bolsa removida;
4     if (!lista_remover(g->estoque, id, &removida))

```

```

5         return 0;
6     op.tipo = OP_SAIDA;
7     op.bolsa = removida;
8     if (!pilha_empilhar(g->historico, op)) {
9         lista_inserir(g->estoque, removida);    /* desfaz a remocao */
10        return 0;
11    }
12    if (saiu != NULL)
13        *saiu = removida;
14    return 1;
15 }
16
17 int estoque_desfazer(GestaoEstoque *g, OperacaoEstoque *desfeita) {
18     OperacaoEstoque op;
19     if (!pilha_desempilhar(g->historico, &op))
20         return 0;
21     if (op.tipo == OP_ENTRADA)
22         lista_remover(g->estoque, op.bolsa.id, NULL);
23     else
24         lista_inserir(g->estoque, op.bolsa);
25     if (desfeita != NULL)
26         *desfeita = op;
27     return 1;
28 }

```

Fonte: Elaborado pelos autores (2026).

Código 5.8 – Registro de saída e desfazer, em Java (GestaoEstoque.java)

```

1 public Bolsa registrarSaida(long id) {
2     Bolsa removida = estoque.remove(id);
3     if (removida == null)
4         return null;
5     historico.empilhar(new OperacaoEstoque(TipoOperacao.SAIDA, removida));
6     return removida;
7 }
8
9 public OperacaoEstoque desfazer() {
10    OperacaoEstoque op = historico.desempilhar();
11    if (op == null)
12        return null;
13    if (op.tipo() == TipoOperacao.ENTRADA)
14        estoque.remove(op.bolsa().getId());
15    else
16        estoque.inserir(op.bolsa());
17    return op;
18 }

```

Fonte: Elaborado pelos autores (2026).

As duas versões seguem a mesma regra: a operação é aplicada primeiro na lista e só é registrada na pilha se tiver ocorrido. Assim, uma entrada com identificador repetido ou uma

saída de bolsa inexistente não geram histórico, o que os testes de ambas as linguagens verificam. Desfazer retira a operação mais recente da pilha e aplica a operação inversa: uma entrada é desfeita removendo a bolsa da lista, e uma saída é desfeita reinserindo a bolsa guardada no histórico.

Para que uma saída possa ser desfeita, a bolsa precisa continuar existindo depois que seu nó deixa a lista. Em C, isso exige uma cópia: `lista_remove` preenche a variável `removida` antes de liberar o nó, e essa cópia é guardada na operação empilhada. Em Java, basta guardar na operação a referência devolvida por `remove`; enquanto a operação estiver na pilha, a bolsa continua alcançável e não é recolhida. A versão em C contém ainda um tratamento que não aparece em Java: se a alocação do nó da pilha falhar, a alteração já feita na lista é desfeita, para que lista e histórico não fiquem inconsistentes. Em Java, a falta de memória interrompe a execução com uma exceção, e não há um retorno nulo a tratar.

5.5 Catálogo: árvore binária de busca

A ABB organiza as bolsas pelo identificador: em cada nó, a subárvore esquerda contém identificadores menores e a direita, maiores. As operações foram escritas de forma recursiva, seguindo um padrão em que a função recebe a raiz de uma subárvore e devolve a nova raiz dessa mesma subárvore, cabendo à chamada anterior religar o ponteiro.

Código 5.9 – Inserção e remoção recursivas na ABB, em C (`arvore_bolsas.c`)

```

1 static NoArvore *inserir_rec(NoArvore *no, Bolsa bolsa, int *resultado) {
2     if (no == NULL) {
3         NoArvore *novo = (NoArvore *) malloc(sizeof(NoArvore));
4         if (novo == NULL) { *resultado = -1; return NULL; }
5         novo->bolsa = bolsa;
6         novo->esquerda = NULL;
7         novo->direita = NULL;
8         *resultado = 1;
9         return novo;
10    }
11    if (bolsa.id < no->bolsa.id)
12        no->esquerda = inserir_rec(no->esquerda, bolsa, resultado);
13    else if (bolsa.id > no->bolsa.id)
14        no->direita = inserir_rec(no->direita, bolsa, resultado);
15    else
16        *resultado = 0;
17    return no;
18 }
19
20 static NoArvore *remover_rec(NoArvore *no, long id, int *removeu) {
21     if (no == NULL)
22         return NULL;
23     if (id < no->bolsa.id)
24         no->esquerda = remover_rec(no->esquerda, id, removeu);
25     else if (id > no->bolsa.id)

```

```

26     no->direita = remover_rec(no->direita, id, removeu);
27 else {
28     NoArvore *filho;
29     if (no->esquerda == NULL || no->direita == NULL) {
30         filho = no->esquerda != NULL ? no->esquerda : no->direita;
31         free(no);
32         *removeu = 1;
33         return filho;
34     }
35     filho = menor_no(no->direita);
36     no->bolsa = filho->bolsa;
37     no->direita = remover_rec(no->direita, filho->bolsa.id, removeu);
38 }
39 return no;
40 }

```

Fonte: Elaborado pelos autores (2026).

Código 5.10 – Inserção e remoção recursivas na ABB, em Java (ArvoreBolsas.java)

```

1 private NoArvore inserirRec(NoArvore no, Bolsa bolsa) {
2     if (no == null) {
3         alterou = true;
4         return new NoArvore(bolsa);
5     }
6     long id = bolsa.getId();
7     long chave = no.bolsa.getId();
8     if (id < chave)
9         no.esquerda = inserirRec(no.esquerda, bolsa);
10    else if (id > chave)
11        no.direita = inserirRec(no.direita, bolsa);
12    return no;
13 }
14
15 private NoArvore removerRec(NoArvore no, long id) {
16     if (no == null)
17         return null;
18     long chave = no.bolsa.getId();
19     if (id < chave)
20         no.esquerda = removerRec(no.esquerda, id);
21     else if (id > chave)
22         no.direita = removerRec(no.direita, id);
23     else {
24         if (no.esquerda == null || no.direita == null) {
25             alterou = true;
26             return no.esquerda != null ? no.esquerda : no.direita;
27         }
28         NoArvore sucessor = menorNo(no.direita);
29         no.bolsa = sucessor.bolsa;
30         no.direita = removerRec(no.direita, sucessor.bolsa.getId());
31     }
32    return no;

```

Fonte: Elaborado pelos autores (2026).

O padrão de devolver a nova raiz da subárvore foi escolhido já na versão em C justamente por se traduzir sem alterações para Java. A alternativa usual em C, que passa um ponteiro para ponteiro (`NoArvore **`) e altera o ponteiro do pai por meio dele, não tem equivalente em Java, linguagem em que não existe ponteiro para variável. Com o padrão adotado, a chamada `no->esquerda = inserir_rec(no->esquerda, ...)` em C e a chamada `no.esquerda = inserirRec(no.esquerda, ...)` em Java são a mesma instrução.

A inserção tem o mesmo caso base nas duas versões: ao alcançar uma subárvore vazia, cria-se o nó e ele é devolvido para ser ligado ao pai. A remoção trata os três casos clássicos na mesma ordem (CORMEN *et al.*, 2012). Um nó sem filhos ou com um único filho é substituído pelo filho (ou por nulo). Um nó com dois filhos recebe a bolsa de seu sucessor em ordem simétrica, que é o menor nó da subárvore direita, e o sucessor é então removido recursivamente dessa subárvore. Os testes das duas versões removem a raiz, que possui dois filhos, e verificam que o sucessor passou a ocupar a raiz.

Duas diferenças decorrem apenas dos recursos de cada linguagem. A primeira é a forma de informar se a operação alterou a árvore: como o retorno da função recursiva já é usado para devolver o nó, a versão em C usa um parâmetro de saída do tipo `int *`; em Java, que não permite ponteiro para inteiro, a mesma informação é registrada no campo privado `alterou`, zerado antes de cada operação pública e consultado ao seu final. A segunda é a liberação dos nós. Em C, o nó removido é liberado com `free` antes de o filho ser devolvido, e a destruição da árvore inteira precisa ocorrer em pós-ordem (filhos antes do pai), porque liberar o pai primeiro tornaria inacessíveis os ponteiros para os filhos. Em Java, basta que o nó deixe de ser referenciado, e a árvore inteira é recolhida quando a referência à raiz se perde.

As demais operações recursivas (busca, percurso em ordem simétrica e cálculo da altura) têm os mesmos casos base e os mesmos passos nas duas linguagens. No percurso, a ação aplicada a cada bolsa é um ponteiro para função acompanhado de um contexto genérico (`void *`) em C e um `Consumer<Bolsa>` em Java.

5.6 Síntese das correspondências

A Tabela 5.1 resume como os mecanismos da versão em C foram representados em Java.

Tabela 5.1 – Correspondência entre os mecanismos das duas implementações

Aspecto	C	Java
Nó da estrutura	struct com ponteiro para o próximo	classe interna privada com referência para o próximo
Criação do nó	malloc e teste de retorno nulo	new; falta de memória gera OutOfMemoryError
Liberação do nó	free explícito, após copiar o necessário	remoção das referências; coletor de lixo
Dado guardado no nó	cópia por valor	referência à entidade JPA
Ausência de resultado	retorno 0 ou NULL e parâmetro de saída	retorno null ou false
Indicador em recursão	parâmetro int *	campo privado alterou
Ação no percurso	ponteiro para função e void *	Consumer<Bolsa>
Destruição da estrutura	função que libera cada nó	desnecessária

Fonte: Elaborado pelos autores (2026).

6 TESTES E RESULTADOS

6.1 Cenários de teste

Os testes foram escritos uma vez em C (`testes.c`) e depois reproduzidos em JUnit 5, na classe `EstruturasU1Test`, com as mesmas entradas e os mesmos resultados esperados. A Tabela 6.1 resume o que foi verificado em cada estrutura.

Tabela 6.1 – Cenários verificados nas duas versões

Estrutura	Cenários
Lista (estoque)	operações na lista vazia; rejeição de identificador duplicado; manutenção da ordem de chegada; remoção no início, no meio e no fim, com atualização de fim; alteração de situação pela referência devolvida na consulta; contagem recursiva
Fila (requisições)	retirada da fila vazia; ordem FIFO; consulta da frente, por identificador e por posição; esvaziamento seguido de nova inserção
Pilha (histórico)	retirada da pilha vazia; ordem LIFO; consulta do topo
Gestão do estoque	entrada duplicada sem registro no histórico; saída de bolsa inexistente; desfazer saída e entrada; desfazer sem histórico
Árvore (catálogo)	operações na árvore vazia; rejeição de duplicado; busca de chave existente e inexistente; percurso em ordem crescente; remoção de folha, de nó com um filho e da raiz com dois filhos; degeneração com chaves inseridas em ordem

Fonte: Elaborado pelos autores (2026).

6.2 Resultados

Na versão em C, as 85 verificações foram aprovadas, tanto na compilação com AddressSanitizer quanto na execução sob o Valgrind. O relatório do Valgrind registrou 43 alocações e 43 liberações, sem nenhum bloco remanescente ao final da execução, o que confirma que todos os nós criados foram liberados. Na versão em Java, os 10 casos de teste da Unidade 1 foram aprovados, assim como a suíte completa da aplicação, com 96 testes. As saídas completas estão na pasta `aed/u1/evidencias` do repositório.

6.3 Custo das operações

Como os algoritmos são os mesmos nas duas linguagens, o custo das operações também é o mesmo. A Tabela 6.2 apresenta esse custo em função de n , o número de elementos, e de h , a altura da árvore.

Tabela 6.2 – Custo assintótico das operações

Estrutura	Inserção	Remoção	Consulta
Lista	$O(n)$	$O(n)$	$O(n)$ por identificador
Fila	$O(1)$	$O(1)$	$O(1)$ na frente; $O(n)$ por identificador
Pilha	$O(1)$	$O(1)$	$O(1)$ no topo
Árvore	$O(h)$	$O(h)$	$O(h)$; percurso completo em $O(n)$

Fonte: Elaborado pelos autores (2026).

Na lista, o encadeamento no fim é feito em tempo constante graças ao ponteiro fim; o custo linear da inserção vem da verificação de identificador duplicado, que percorre a lista. Na árvore, h corresponde a $\log_2 n$ quando ela está equilibrada, mas pode chegar a n : um dos testes insere os identificadores de 1 a 10 em ordem crescente e obtém altura 10, pois cada novo nó é ligado sempre à direita do anterior. O balanceamento da árvore não faz parte do escopo desta unidade.

7 INTEGRAÇÃO COM A APLICAÇÃO

As classes Java não dependem do Spring, de anotações ou do banco de dados. Elas recebem e devolvem as entidades Bolsa e Requisicao já usadas pelos serviços da aplicação, e podem ser instanciadas diretamente por um serviço ou registradas como componentes do contexto. O Código 7.1 mostra um exemplo de serviço que carrega as bolsas persistidas e expõe o estoque, o desfazer e o catálogo ordenado.

Código 7.1 – Exemplo de uso das estruturas por um serviço da aplicação

```

1 @Service
2 public class PaineEstoqueService {
3
4     private final GestaoEstoque gestao = new GestaoEstoque();
5     private final ArvoreBolsas catalogo = new ArvoreBolsas();
6
7     public PaineEstoqueService(BolsaRepository bolsas) {
8         bolsas.findAll().forEach(b -> {
9             gestao.registrarEntrada(b);
10            catalogo.inserir(b);
11        });
12    }
13
14    public List<Bolsa> estoqueAtual()      { return gestao.getEstoque().paraLista();
15    }
16    public OperacaoEstoque desfazer()      { return gestao.desfazer(); }
17    public List<Bolsa> catalogoOrdenado() { return catalogo.paraListaOrdenada(); }
18 }

```

Fonte: Elaborado pelos autores (2026).

Os métodos `paraLista` e `paraListaOrdenada` existem apenas para entregar os dados à camada web, onde são convertidos em JSON, e devolvem uma cópia. Eles não são usados pelas estruturas internamente. As classes não são seguras para acesso concorrente; caso sejam compartilhadas entre requisições HTTP simultâneas, o serviço que as utiliza deve sincronizar o acesso.

8 CONSIDERAÇÕES FINAIS

As estruturas da Unidade 1 foram implementadas em C, com alocação e liberação explícitas de memória, e reescritas em Java sem perder a correspondência com o código original. A comparação trecho a trecho mostrou que os algoritmos são os mesmos nas duas linguagens e que as diferenças se concentram no gerenciamento de memória e na forma de sinalizar resultados. Em C, a liberação explícita impõe cuidados que não aparecem em Java, como copiar o dado antes de liberar o nó, recolocar o ponteiro de fim em nulo ao esvaziar a fila e destruir a árvore em pós-ordem. Em Java, esses pontos são resolvidos pelo coletor de lixo, mas a lógica de religação das referências precisa ser a mesma, como mostrou o caso da fila.

Do ponto de vista do projeto, as classes Java já estão prontas para serem usadas pelos serviços da aplicação. Na Unidade 2, as estruturas servirão de base para a priorização por validade, o índice por *hash*, a compatibilidade ABO/Rh e a roteirização, que serão implementados com a mesma dinâmica de versão em C, versão em Java e tradução comentada.

REFERÊNCIAS

CORMEN, T. H. *et al.* **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

KERNIGHAN, B. W.; RITCHIE, D. M. **C: a linguagem de programação padrão ANSI**. Rio de Janeiro: Campus, 1989.

ORACLE. **The Java Language Specification**: Java SE 17 Edition. [S. l.]: Oracle, 2021.

Disponível em: <https://docs.oracle.com/javase/specs/jls/se17/html/>. Acesso em: 1 out. 2026.

TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. **Estruturas de dados usando C**. São Paulo: Makron Books, 1995.

ZIVIANI, N. **Projeto de algoritmos: com implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2011.

APÊNDICE A – ORGANIZAÇÃO DOS ARQUIVOS E EXECUÇÃO

O código desta entrega está distribuído no repositório conforme o Quadro A.1.

Quadro A.1 – Arquivos da entrega

Arquivo	Conteúdo
aed/u1/c/dominio.h, dominio.c	tipos do domínio
aed/u1/c/lista_estoque.h, .c	lista do estoque
aed/u1/c/fila_requisicoes.h, .c	fila de requisições
aed/u1/c/pilha_historico.h, .c	pilha do histórico
aed/u1/c/gestao_estoque.h, .c	lista e pilha integradas
aed/u1/c/arvore_bolsas.h, .c	árvore binária de busca
aed/u1/c/testes.c, demo.c, Makefile	testes, demonstração e compilação
rotavital/src/main/java/ com/hemorede/estruturas/	ListaEstoque, FilaRequisicoes, PilhaHistorico, OperacaoEstoque, TipoOperacao, GestaoEstoque, ArvoreBolsas
rotavital/src/test/java/ com/hemorede/estruturas/	EstruturasUITest
aed/u1/evidencias/	saídas dos testes e do Valgrind

Fonte: Elaborado pelos autores (2026).

Para compilar e testar a versão em C, a partir da pasta aed/u1/c, usam-se os comandos `make test` (testes com AddressSanitizer), `make valgrind` (testes e demonstração sob o Valgrind) e `make run` (demonstração com dados sintéticos). No Windows, com o MinGW, os testes podem ser compilados com `gcc -std=c99 -o testes.exe testes.c dominio.c lista_estoque.c fila_requisicoes.c pilha_historico.c gestao_estoque.c arvore_bolsas.c`. Para a versão em Java, a partir da pasta rotavital, usa-se `mvn test -Dtest=EstruturasUITest`.